

✓ Project: Smart Fraud Detection Pipeline

Bronze Layer Ingestion

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopargaon

Email: snehalbhosale1807@gmail.com

Student ID: CT_CSI_DE_1177 **Technology:** PySpark, Spark SQL, Parquet/Delta Lake

Objective:

To ingest raw CSV files without modifying business data and preserve source information for traceability.

✓ Step 1: Import Libraries

```
import os
import pandas as pd

from pyspark.sql import SparkSession
from pyspark.sql.functions import current_timestamp, lit
```

✓ Step 2: Upload Files

```
from google.colab import files

uploaded = files.upload()
```

3 files selected.

accounts.csv(text/csv) - 21976 bytes, last modified: n/a - 100% done

fraud_watchlist.csv(text/csv) - 1576 bytes, last modified: n/a - 100% done

transactions.csv(text/csv) - 963914 bytes, last modified: n/a - 100% done

Saving accounts.csv to accounts.csv

Saving fraud_watchlist.csv to fraud_watchlist.csv

Saving transactions.csv to transactions.csv

```
print(os.listdir())
```

```
['.config', 'fraud_watchlist.csv', 'accounts.csv', 'transactions.csv', 'sampl
```

✓ Step 3: Verify the Files

```
required_files = [  
    "accounts.csv",  
    "transactions.csv",  
    "fraud_watchlist.csv"  
]  
  
for file in required_files:  
    if os.path.exists(file):  
        print(f"✓ {file} found")  
    else:  
        print(f"X {file} not found")
```

```
✓ accounts.csv found  
✓ transactions.csv found  
✓ fraud_watchlist.csv found
```

✓ Step 4: Define paths

```
accounts_path = "/content/accounts.csv"  
transactions_path = "/content/transactions.csv"  
fraud_path = "/content/fraud_watchlist.csv"
```

✓ Step 5: Start Spark

```
spark = (  
    SparkSession.builder  
        .appName("SmartFraudDetection")  
        .getOrCreate()  
)  
  
print("Spark Version:", spark.version)
```

```
Spark Version: 4.0.3
```

✓ Step 6: Read Raw CSV Files

```
accounts_raw = (  
    spark.read  
        .option("header", True)  
        .option("inferSchema", False)  
        .csv(accounts_path)  
)
```

```

)

transactions_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(transactions_path)
)

fraud_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(fraud_path)
)

```

✓ Step 7: Check Row Counts

```

print("Accounts:", accounts_raw.count())
print("Transactions:", transactions_raw.count())
print("Fraud Watchlist:", fraud_raw.count())

```

```

Accounts: 505
Transactions: 20011
Fraud Watchlist: 46

```

✓ Step 8: Display Raw Data

```

print("ACCOUNTS")
accounts_raw.show(5, truncate=False)

print("TRANSACTIONS")
transactions_raw.show(5, truncate=False)

print("FRAUD WATCHLIST")
fraud_raw.show(5, truncate=False)

```

ACCOUNTS

account_id	customer_name	account_type	credit_limit	branch
ACC0059	Customer_59	Salary	500000	Pune
ACC0139	Customer_139	Current	100000	Delhi
ACC0182	Customer_182	Salary	200000	Pune
ACC0302	Customer_302	Savings	500000	Mumbai
ACC0254	Customer_254	Savings	200000	Delhi

only showing top 5 rows

TRANSACTIONS

txn_id	account_id	txn_date	amount	merchant
TXN009314	ACC0332	2025-01-10	107549.32	Swiggy
TXN013000	ACC0412	2025-03-17	22297.68	Zomato
TXN018018	ACC0486	2025-05-10	115208.63	Zomato
TXN013757	ACC0443	2025-07-24	45099.7	ATM-Withdrawal
TXN016494	ACC0062	2025-02-25	70884.04	Swiggy

only showing top 5 rows

FRAUD WATCHLIST

account_id	fraud_type	flagged_date
ACC0116	Account Takeover	2025-05-26
ACC0300	Card Skimming	10-Feb-2025
ACC0280	Account Takeover	2025-03-03
ACC0464	Account Takeover	2025-07-14
ACC0072	Account Takeover	2025-05-17

only showing top 5 rows

Step 9: Check Raw Schema

```
print("Accounts Schema")
accounts_raw.printSchema()

print("\nTransactions Schema")
transactions_raw.printSchema()

print("\nFraud Watchlist Schema")
fraud_raw.printSchema()
```

Accounts Schema

root

```
-- account_id: string (nullable = true)
-- customer_name: string (nullable = true)
-- account_type: string (nullable = true)
-- credit_limit: string (nullable = true)
-- branch: string (nullable = true)
```

Transactions Schema

root

```
-- txn_id: string (nullable = true)
-- account_id: string (nullable = true)
-- txn_date: string (nullable = true)
-- amount: string (nullable = true)
-- merchant: string (nullable = true)
```

Fraud Watchlist Schema

```

fraud_watchlist schema
root
|-- account_id: string (nullable = true)
|-- fraud_type: string (nullable = true)
|-- flagged_date: string (nullable = true)

```

✓ Step 10: Add Audit Metadata

```

accounts_bronze = (
    accounts_raw
    .withColumn(
        "ingestion_timestamp",
        current_timestamp()
    )
    .withColumn(
        "source_system",
        lit("accounts.csv")
    )
)

transactions_bronze = (
    transactions_raw
    .withColumn(
        "ingestion_timestamp",
        current_timestamp()
    )
    .withColumn(
        "source_system",
        lit("transactions.csv")
    )
)

fraud_bronze = (
    fraud_raw
    .withColumn(
        "ingestion_timestamp",
        current_timestamp()
    )
    .withColumn(
        "source_system",
        lit("fraud_watchlist.csv")
    )
)

```

✓ Step 11: Create Project Folders

```
import os
```

```
import os

os.makedirs("/content/data/processed/bronze", exist_ok=True)
os.makedirs("/content/data/processed/silver", exist_ok=True)
os.makedirs("/content/data/processed/gold", exist_ok=True)

print("Project directories created successfully.")
```

Project directories created successfully.

✓ Step 12: Write Bronze Data

```
accounts_bronze.write \
    .mode("overwrite") \
    .parquet("/content/data/processed/bronze/accounts")

transactions_bronze.write \
    .mode("overwrite") \
    .parquet("/content/data/processed/bronze/transactions")

fraud_bronze.write \
    .mode("overwrite") \
    .parquet("/content/data/processed/bronze/fraud_watchlist")
```

✓ Step 13: Read Bronze Data

```
accounts_bronze_check = spark.read.parquet(
    "/content/data/processed/bronze/accounts"
)

transactions_bronze_check = spark.read.parquet(
    "/content/data/processed/bronze/transactions"
)

fraud_bronze_check = spark.read.parquet(
    "/content/data/processed/bronze/fraud_watchlist"
)
```

✓ Step 14: Bronze Validation

```
print("Bronze Accounts:", accounts_bronze_check.count())
print("Bronze Transactions:", transactions_bronze_check.count())
print("Bronze Fraud Watchlist:", fraud_bronze_check.count())
```

Bronze Accounts: 505
 Bronze Transactions: 30011
 Bronze Fraud Watchlist: 100

```
bronze_transactions: 20011
Bronze Fraud Watchlist: 46
```

✓ Step 15: Verify Dirty Data Is Preserved

Check missing account ID

```
transactions_bronze_check.filter(
    transactions_bronze_check.account_id.isNull()
).show(truncate=False)
```

txn_id	account_id	txn_date	amount	merchant	ingestion_timestamp
TXN900001	NULL	2025-03-01	4500.0	Amazon	2026-08-08 17:02:47.406351

Check "N/A" amount

```
transactions_bronze_check.filter(
    transactions_bronze_check.amount == "N/A"
).show(truncate=False)
```

txn_id	account_id	txn_date	amount	merchant	ingestion_timestamp
TXN900007	ACC0060	2025-02-12	N/A	BigBasket	2026-08-08 17:02:47.406351

Check orphan account

```
transactions_bronze_check.filter(
    transactions_bronze_check.amount == "N/A"
).show(truncate=False)
```

txn_id	account_id	txn_date	amount	merchant	ingestion_timestamp
TXN900007	ACC0060	2025-02-12	N/A	BigBasket	2026-08-08 17:02:47.406351

✓ Step 16: Download the Bronze Dataset:


```
from google.colab import files

# Save Bronze datasets as CSV
accounts_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("bronze_accounts_csv")

transactions_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("bronze_transactions_csv")

fraud_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("bronze_fraud_watchlist_csv")
```

```
import shutil
from google.colab import files

# Temporary folders
accounts_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("temp_accounts")

transactions_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("temp_transactions")

fraud_bronze.coalesce(1).write.mode("overwrite") \
    .option("header", True) \
    .csv("temp_fraud")

# Rename Spark-generated CSV files
def get_csv_file(folder):
    import glob
    return glob.glob(f"{folder}/part-*.csv")[0]

shutil.copy(
    get_csv_file("temp_accounts"),
    "bronze_accounts.csv"
)

shutil.copy(
    get_csv_file("temp_transactions"),
    "bronze_transactions.csv"
)

shutil.copy(
    get_csv_file("temp_fraud"),
    "bronze_fraud_watchlist.csv"
)
```

```
)  
  
# Download  
files.download("bronze_accounts.csv")  
files.download("bronze_transactions.csv")  
files.download("bronze_fraud_watchlist.csv")
```

✓ **Conclusion

The three raw datasets were successfully ingested into the Bronze layer using PySpark.

The raw data was preserved without applying transformations. Ingestion timestamp and source-system metadata were added to maintain data lineage and traceability.

The Bronze layer contains:

- 505 account records
- 20,011 transaction records
- 46 fraud watchlist records

Intentional inconsistencies were preserved and will be handled during Silver-layer data cleaning and validation.

The Bronze layer was successfully implemented using PySpark in Google Colab. Raw account, transaction and fraud watchlist data was ingested and stored in Parquet format with audit metadata. The raw data quality issues were intentionally preserved, providing a reliable input for the Silver-layer cleaning and validation process.

Start coding or [generate](#) with AI.

